

Teste Técnico para Desenvolvedor Junior: Mini Roteirizador de Entregas

1. Introdução

Este teste técnico visa avaliar as competências de um candidato a Desenvolvedor Junior, com foco em **TypeScript, Ruby, Vue.js e Docker**. O desafio proposto é a construção de um sistema simplificado de roteirização de entregas, onde **todos os componentes (frontend, microserviços e bancos de dados) devem ser orquestrados via Docker Compose**, permitindo ao candidato demonstrar sua capacidade de projetar, implementar e integrar diferentes componentes de software em um domínio de negócio relevante para a empresa.

2. Cenário de Negócio: Mini Roteirizador de Entregas

Você trabalhará no desenvolvimento de um sistema básico para gerenciar entregas. O objetivo é permitir que uma empresa de logística cadastre pontos de entrega (endereços) e veículos, e então visualize esses pontos e associe veículos a eles para simular uma rota de entrega. O foco não é na otimização complexa de rotas, mas na gestão e visualização dos elementos envolvidos.

3. Arquitetura Proposta

A solução deverá ser implementada utilizando uma arquitetura de microserviços, **totalmente containerizada com Docker e orquestrada via Docker Compose**, composta por:

- **Frontend:** Uma aplicação web desenvolvida com Vue 3.
- **Microserviço de Gestão (Ruby):** Uma API RESTful desenvolvida em Ruby (preferencialmente Ruby on Rails com ActiveRecord), responsável por gerenciar entidades como endereços, veículos e motoristas.
- **Microserviço de Roteirização (TypeScript):** Uma API RESTful desenvolvida em TypeScript (Node.js), responsável por processar e calcular informações relacionadas a rotas, como distâncias ou ordenação de paradas.

4. Requisitos Técnicos

4.1. Frontend (Vue 3)

O frontend deve ser uma Single Page Application (SPA) que:

- Exiba uma lista de endereços de entrega, mostrando detalhes como rua, número, cidade, estado e status (ex: 'pendente', 'em rota', 'entregue').
- Exiba uma lista de veículos disponíveis, mostrando placa, capacidade.
- Permita associar um veículo a um ou mais endereços, simulando a atribuição de uma rota. Esta associação deve ser visualmente clara (ex: agrupando endereços por veículo).
- Consuma os dados corretamente entre o ruby e o typescript.
- Utilize TypeScript para tipagem e melhor organização do código.

4.2. Microserviço de Gestão (Ruby)

Este microserviço deve ser uma API RESTful desenvolvida em Ruby (preferencialmente utilizando Ruby on Rails com ActiveRecord para o ORM), responsável por gerenciar as entidades principais do sistema de roteirização. Ele deve:

- Gerenciar **Endereços** com os seguintes atributos: `id` (UUID), `rua` (string), `numero` (string), `cidade` (string), `estado` (string), `cep` (string), `latitude` (decimal), `longitude` (decimal).
- Gerenciar **Veículos** com os seguintes atributos: `id` (UUID), `placa` (string), `capacidade` (inteiro, ex: número de pacotes), `modelo` (string).
- Gerenciar **Motoristas** com os seguintes atributos: `id` (UUID), `nome` (string), `cpf` (string).
- Expor endpoints RESTful para CRUD (Create, Read, Update, Delete) de cada uma dessas entidades (Endereços, Veículos, Motoristas).
- Utilizar um banco de dados **PostgreSQL** ou **MySQL** (via Docker).

4.3. Microserviço de Roteirização (TypeScript)

Este microserviço deve ser uma API RESTful desenvolvida em TypeScript (Node.js, utilizando um framework como Express.js), responsável por funcionalidades de processamento e cálculo. Ele deve:

- Receber uma lista de endereços (IDs) e um veículo (ID) do frontend.
- Expor os seguintes endpoints:
 - `POST /rotas/calcular` : Recebe `veiculo_id` e uma lista de `endereço_ids`, e retorna uma ordem sugerida de paradas.
 - `POST /rotas/atribuir` : Recebe `veiculo_id` e uma lista de `endereço_ids` ordenados, e atualiza o status dos endereços no Microserviço de Gestão (via chamada HTTP interna ou simplesmente simulando a atualização).

- Utilizar um banco de dados **PostgreSQL** ou **MySQL** (via Docker) para armazenar informações de rotas calculadas ou atribuídas (opcional, mas um diferencial).

Implementação do Cálculo de Rotas

O cálculo de rotas não precisa implementar algoritmos avançados de otimização logística. Para simplificação, o candidato poderá:

- Ordenar os endereços por latitude/longitude;
- Retornar uma ordem aleatória;
- Implementar uma lógica simples de proximidade; ou
- Integrar uma API externa gratuita de roteirização/geolocalização.

Exemplos de serviços que podem ser utilizados:

- OpenRouteService;
- OSRM;
- APIs baseadas em OpenStreetMap.

O objetivo desta etapa é avaliar:

- Integração entre serviços;
- Organização da arquitetura;
- Consumo de APIs;
- Manipulação de dados;
- Clareza da implementação.

A implementação de algoritmos complexos de otimização de rotas não é obrigatória.

4.4. Comunicação entre Serviços

- O frontend deve se comunicar diretamente com o Microserviço de Gestão e o Microserviço de Roteirização, ou o frontend, com o de gestão e o de gestão com o de roteirização.
- O Microserviço de Roteirização tem que se comunicar com o Microserviço de Gestão para atualizar o status dos endereços após uma rota ser atribuída. Esta comunicação deve ser feita via chamadas HTTP internas.
- A integração da informação (ex: mostrar detalhes do veículo e endereços atribuídos a uma rota) deve ser feita no frontend.

4.5. Persistência de Dados e Orquestração

- Cada microserviço deve ter seu próprio banco de dados isolado ou apenas o ruby e o de roteirização devolvendo os dados para o ruby. Para ambos, utilize **PostgreSQL** ou **MySQL**.
- **Orquestração Completa com Docker Compose:** Toda a aplicação (frontend, microserviços e seus respectivos bancos de dados) deve ser containerizada e orquestrada via **Docker Compose**. O candidato deve fornecer um arquivo `docker-compose.yml` funcional que permita subir e configurar todo o ambiente com um único comando: `docker-compose up`. Isso inclui a configuração da comunicação entre os serviços e a persistência dos dados nos bancos de dados containerizados.
- O uso de ORMs (ex: ActiveRecord para Ruby, TypeORM/Prisma para TypeScript) é encorajado.

5. Critérios de Avaliação

O teste será avaliado com base nos seguintes critérios:

- **Funcionalidade:** A aplicação funciona conforme os requisitos? Todos os endpoints e funcionalidades estão implementados corretamente? Comunicação entre os 3 códigos fontes?
- **Qualidade do Código:** Organização, legibilidade, clareza, aderência a boas práticas de programação (Clean Code, SOLID, etc.).
- **Uso de Tecnologias:** Correta aplicação de TypeScript, Vue 3 e Ruby. Demonstração de conhecimento dos frameworks e linguagens.
- **Tratamento de Erros:** Como a aplicação lida com erros (ex: recurso não encontrado, dados inválidos).
- **Testes:** Implementação de testes unitários ou de integração (opcional, mas um diferencial).
- **Documentação:** `README.md` claro com instruções para rodar a aplicação, descrição da arquitetura e decisões tomadas.
- **Versionamento:** Uso adequado do Git (commits claros e concisos).

6. Instruções para Entrega

1. Crie um repositório Git (pode ser público no GitHub, GitLab, etc.).
2. Organize o projeto em diretórios separados para o frontend e cada microserviço.
3. **Configuração Docker:** Inclua todos os `Dockerfile`s necessários para cada serviço e um arquivo `docker-compose.yml` na raiz do projeto que orquestre a subida de todos os

serviços (frontend, microserviços e bancos de dados) com um único comando `docker-compose up`.

4. Inclua um arquivo `README.md` na raiz do repositório com:

- Uma breve descrição do projeto.
- **Instruções Claras de Setup com Docker Compose:** Detalhes sobre como configurar e rodar a aplicação utilizando `docker-compose up`.
- Quaisquer decisões de design ou escolhas técnicas relevantes.
- Considerações sobre melhorias futuras (opcional).

1. Compartilhe o link do repositório Git para avaliação.